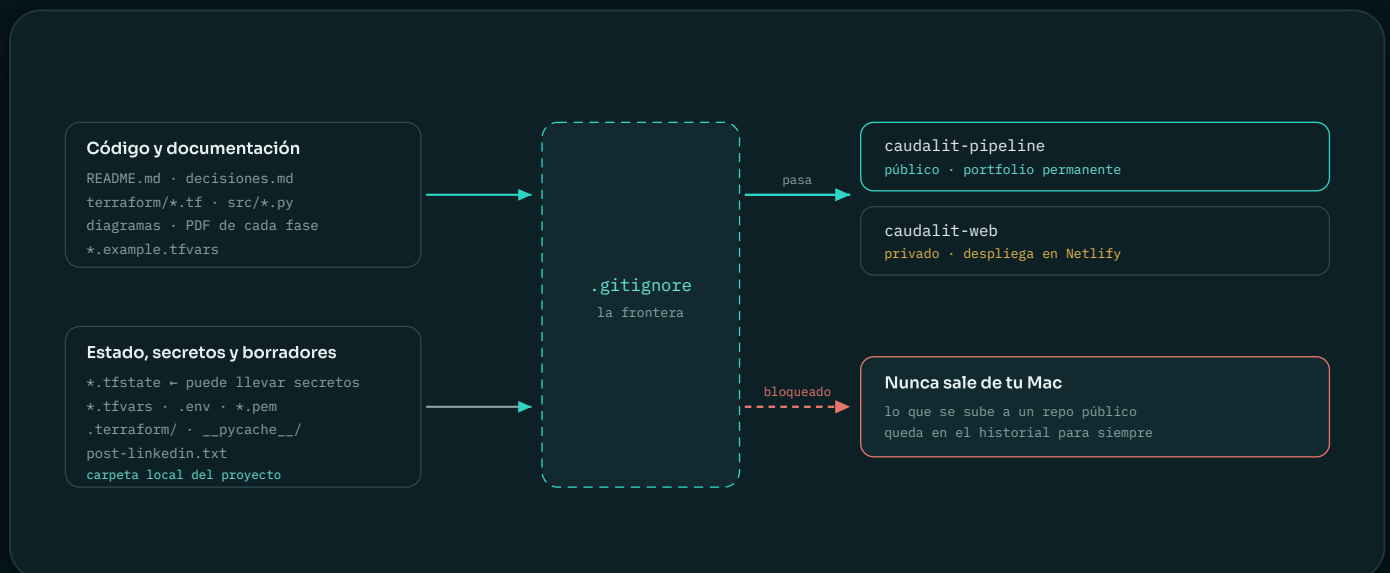


El archivo más importante no tiene código.

La Fase 2 no despliega nada ni procesa un solo dato. Decide algo más difícil de deshacer: qué sale de tu ordenador y qué se queda dentro. En un repositorio público, esa frontera solo se cruza en un sentido.

La frontera

Todo lo del proyecto vive en la misma carpeta. El `.gitignore` es lo único que decide qué parte de ella se hace pública.



El `.gitignore` se escribe **antes** del primer commit, no después. Un archivo que ya entró en el historial sigue ahí aunque lo borres en un commit posterior: para sacarlo hay que reescribir la historia entera del repositorio.

Dos repositorios, no uno

Cambian a ritmos distintos y se despliegan distinto. Juntarlos habría significado que cada corrección de una coma en la web ensuciara el historial del proyecto técnico.

PÚBLICO

caudalit-pipeline

El proyecto de datos y su documentación. Es la pieza de portfolio: cuando al final se ejecute `terraform destroy` y la infraestructura desaparezca, esto es lo que queda como prueba del trabajo.

PRIVADO

caudalit-web

La landing de caudalit.com. Privado porque el código fuente de un sitio comercial no aporta nada al portfolio. Conectado a Netlify: cada `push` a `main` despliega solo.

Qué se excluye, y por qué

Cada regla del `.gitignore` responde a un riesgo concreto, no a una costumbre heredada.

REGLA	MOTIVO
<code>*.tfstate</code>	Terraform guarda ahí el estado real de la infraestructura, y puede incluir valores sensibles en texto plano . Es la filtración más habitual en repos de IaC.
<code>*.tfvars</code>	Los valores reales de las variables: IDs de cuenta, ARNs, nombres de bucket. Se versiona solo <code>*.example.tfvars</code> con valores de mentira.
<code>.terraform/</code>	Providers descargados. Cientos de megas que se regeneran con <code>terraform init</code> .
<code>.env</code> · <code>*.pem</code> · <code>*.key</code>	Credenciales y claves privadas. Nunca, bajo ningún concepto.
<code>post-linkedin.txt</code>	Los borradores de las publicaciones. No son parte del proyecto técnico, y en un repo de portfolio desdibujan lo que se está enseñando.
<code>__pycache__/</code> · <code>*.zip</code>	Artefactos de compilación y empaquetado de las Lambdas. Se regeneran.
<code>.DS_Store</code>	Metadatos de Finder. Ruido puro, y delata la estructura de carpetas de tu máquina.

Las decisiones de esta fase

01

Correo privado en los commits

GitHub ofrece una dirección `@users.noreply`. Sin ella, tu correo real queda escrito en cada commit de un repositorio público, y hay bots rastreando GitHub precisamente para eso. Quitarlo después obliga a reescribir todo el historial.

02

Un commit por fase

El historial cuenta la misma historia que las publicaciones. Quien abra el repositorio dentro de un año puede leer cómo evolucionó la decisión, no solo dónde acabó.

03

El README es la portada

Abre con el diagrama y la pregunta de negocio, no con la lista de tecnologías. Quien entra decide en cinco segundos si sigue leyendo, y «¿cuánto sube la luz por cada grado?» retiene más que «Terraform, Glue, PySpark».

04

Estructura completa desde el primer commit

Las carpetas vacías no existen en Git. Con un `.gitkeep` el repositorio ya enseña hacia dónde va — `terraform/`, `src/`, `athena/` — aunque solo esté hecha la primera fase.

• Un repositorio público solo se cruza en un sentido

Borrar un archivo no lo borra del historial. Sigue accesible en el commit donde entró, y GitHub y los buscadores pueden haberlo cacheado. Sacarlo de verdad exige reescribir la historia del repositorio y forzar el `push`, que rompe cualquier copia que alguien haya clonado.

Por eso la revisión va **antes** del primer `push`, no después: se comprueba qué archivos entran, se verifica que no hay estado ni credenciales, y solo entonces se sube.